

TASK 12 – RE-EVALUATE AND ADAPT THE DESIGN

Changed Requirement

The university now needs to process **50,000 search requests per hour** on a database containing **1,000,000 student records**. New student records are added only **once per day**.

The change in requirements makes search performance much more important than before. Since the database is very large and search requests are frequent while updates are relatively rare, the algorithm should minimise the cost of each individual search.

A. IMPACT ANALYSIS

1. What has changed from the original problem?

Initially, the system was required to locate student registration numbers as the database increased from approximately **1,000 to 10,000, 100,000 and 1,000,000+ records**.

Under the new requirement:

- The database contains exactly **1,000,000 student records**.
- The system must process **50,000 search requests per hour**.
- New records are added only **once per day**.
- Therefore, the number of searches is very high compared with the frequency of database updates.

The system now needs to be optimised mainly for **fast repeated searching**.

2. Which original assumptions are affected?

The original design considered Linear Search, Recursive Binary Search and Hash Table Search for different database sizes.

The new requirement changes the importance of preprocessing and search time. A preprocessing operation that is performed once can now be justified if it significantly reduces the cost of thousands of subsequent searches.

For example, sorting one million records or constructing a hash table may require additional processing time, but this cost can be recovered through the large number of searches performed afterwards.

3. Why is search frequency important?

The system performs:

50,000 searches/hour

Therefore, in one day:

$$50,000 \times 24 = 1,200,000$$

So the system may process approximately:

1,200,000 searches/day

This means that even a small increase in the cost of one search can become significant when multiplied by more than one million searches per day.

Therefore, an algorithm with a low search complexity is highly desirable.

4. Why does the low update frequency influence the design?

New student records are added only once per day. Therefore, the data structure does not need to be rebuilt continuously.

A relatively expensive preprocessing operation can be performed once after the daily update. The resulting structure can then be used for a large number of searches.

This makes **hash table construction** particularly attractive because the preprocessing cost can be distributed across many search operations.

B. EVALUATION OF THE ORIGINAL DESIGN

The three original approaches can now be evaluated under the new conditions.

1. Linear Search

Linear Search has a worst-case complexity of:

$$O(n)$$

For:

$$n = 1,000,000$$

a search may require checking up to approximately:

$$1,000,000$$

records.

For 50,000 searches per hour, the worst-case number of comparisons could be approximately:

$$\begin{aligned} &50,000 \times 1,000,000 \\ &= 50,000,000,000 \end{aligned}$$

comparisons per hour.

Therefore, Linear Search is not suitable for the new requirement.

2. Recursive Binary Search

Binary Search has a search complexity of:

$$O(\log_2 n)$$

For one million records:

$$\log_2(1,000,000) \approx 19.93$$

Therefore, approximately **20 comparisons** are required in the worst case for one search.

For 50,000 searches:

$$\begin{aligned} &50,000 \times 20 \\ &= 1,000,000 \end{aligned}$$

approximately one million comparisons per hour.

This is much better than Linear Search.

However, Binary Search requires the registration numbers to be **sorted**. Sorting introduces an additional preprocessing cost.

3. Hash Table Search

Hashing provides an expected search complexity of approximately:

$$O(1)$$

when the hash function distributes the registration numbers efficiently and the table has a suitable load factor.

The hash table must first be constructed, which creates a preprocessing cost.

However, because records are updated only once per day, this preprocessing cost does not occur for every search.

After construction, the same hash table can serve a very large number of search requests.

Original Design Evaluation

Algorithm	Search Complexity	Suitability for 1,000,000 records
Linear Search	$O(n)$	Poor
Binary Search	$O(\log n)$	Very good
Hash Search	$O(1)$ expected	Excellent

Therefore, Linear Search should be rejected for the new requirement.

Binary Search remains a strong option, but Hash Search is more suitable because the system performs a very large number of searches while updates occur only once per day.

C. ADAPTATION OR REDESIGN

Final Decision: MODIFY THE ORIGINAL DESIGN

The original design will be **modified** rather than completely discarded.

The recommended revised strategy is:

Daily preprocessing

At the time of the daily database update:

1. Read all student registration numbers.
2. Construct or update the hash table.
3. Handle collisions using an appropriate collision-resolution technique.
4. Keep the resulting hash table available for the day's searches.

During normal operation

For every search request:

1. Receive the required registration number.
2. Apply the hash function.
3. Calculate the corresponding table index.
4. Check the location.
5. If a collision exists, follow the collision-resolution procedure.
6. Return the student record if the registration number is found.

Revised Strategy

Daily Database Update



Construct / Update Hash Table



Hash Function



Hash Table Ready



50,000 Search Requests/Hour



Calculate Hash Index



Search Corresponding Location



Return Student Record

Effect on Computational Complexity

Hash table construction requires approximately:

$$O(n)$$

time for inserting n records, assuming efficient hashing.

For:

$$n = 1,000,000$$

the preprocessing is approximately:

$$O(1,000,000)$$

However, this operation is performed only around once per day.

The expected search cost is:

$$O(1)$$

per search.

Therefore, for q searches, the overall cost can be represented as:

$$T(n, q) = O(n) + O(q)$$

where:

- n = number of student records
- q = number of search requests

For one day:

$$n = 1,000,000$$

and

$$q = 1,200,000$$

Therefore:

$$T(n, q) = O(1,000,000) + O(1,200,000)$$

which is approximately:

$$O(2,200,000)$$

in terms of the combined high-level operation counts, under the expected constant-time hashing assumption.

This is much more suitable than repeatedly scanning one million records.

D. EVIDENCE-BASED DEFENCE

The decision to modify the original design and use a **hash-table-based search system** is supported by the mathematical and operational requirements.

1. Mathematical Complexity

Linear Search:

$$O(n)$$

Binary Search:

$$O(\log n)$$

Hash Search:

$$O(1) \text{ expected}$$

With:

$$n = 1,000,000$$

Linear Search may examine up to one million records for a single unsuccessful search.

Binary Search requires approximately:

$$\log_2(1,000,000) \approx 20$$

comparisons.

Hash Search is expected to locate the record with a constant number of operations when collisions are controlled.

2. Search Frequency

The system processes:

50,000 searches/hour

or:

1,200,000 searches/day

Because the number of searches is extremely high, reducing the cost of each search has a major effect on total system performance.

3. Preprocessing Cost

Hashing requires the construction of a hash table.

Although this introduces an $O(n)$ preprocessing cost, the table can then be reused for a large number of searches.

Since records are added only once per day, the preprocessing does not have to be repeated after every search.

Therefore, the preprocessing cost is acceptable.

4. Dataset Size

The database contains:

1,000,000

records.

At this size, Linear Search becomes impractical for frequent requests.

Hashing is more suitable because it avoids scanning the complete database for every search.

5. Computational Trade-offs

Hashing also has some limitations:

- Additional memory is required.
- Hash collisions may occur.
- A poor hash function can reduce performance.
- The hash table must be updated when new records are added.
- Performance depends on maintaining a suitable load factor.

However, these disadvantages are acceptable because the database is updated only once per day while the number of searches is very high.

FINAL RECOMMENDATION

The recommended solution is to **MODIFY the original design** by making the **Hash Table Search** the primary searching strategy.

Linear Search should not be used for the production system because its $O(n)$ search cost is too high for one million records and 50,000 searches per hour.

Recursive Binary Search remains a strong alternative with $O(\log n)$ search complexity, but it requires sorted data.

Hash Search is preferred because it provides **expected $O(1)$ search time** after preprocessing. The one-time daily preprocessing cost is reasonable because the resulting hash table can handle a very large number of searches throughout the day.

Therefore, considering the **1,000,000-record dataset, 50,000 searches per hour, approximately 1.2 million searches per day, low update frequency, preprocessing cost and computational trade-offs**, the hash-table-based design is the most appropriate solution for the revised student-record system.